

MAPS — Инструкция администратора

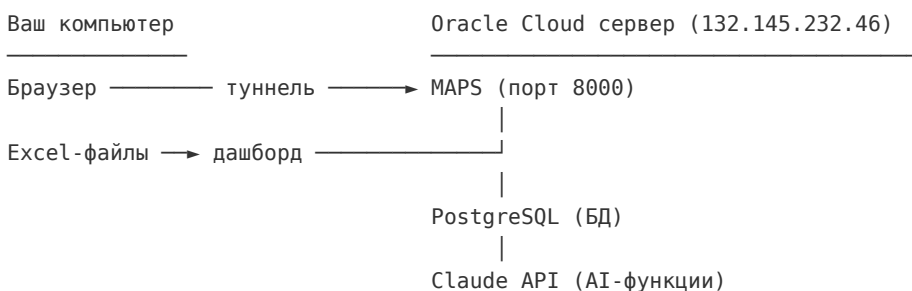
Для кого: Начинающий разработчик или администратор системы.

Что внутри: Как запустить, как работать ежедневно, что делать когда что-то сломалось.

Содержание

1. [Как устроена система](#1-как-устроена-система)
2. [Запуск на локальном компьютере](#2-запуск-на-локальном-компьютере)
3. [Работа с сервером Oracle Cloud](#3-работа-с-сервером-oracle-cloud)
4. [Ежедневная работа: полный цикл](#4-ежедневная-работа-полный-цикл)
5. [Загрузка данных подробно](#5-загрузка-данных-подробно)
6. [Запуск распределения и скачивание отчёта](#6-запуск-распределения-и-скачивание-отчёта)
7. [AI-функции](#7-ai-функции)
8. [Обновление кода на сервере](#8-обновление-кода-на-сервере)
9. [Тестовые данные и переход на реальные](#9-тестовые-данные-и-переход-на-реальные)
10. [Работа с GitHub](#10-работа-с-github)
11. [Типичные ошибки и как их исправить](#11-типичные-ошибки-и-как-их-исправить)
12. [Работа через командную строку (CLI)](#12-работа-через-командную-строку-cli)
13. [Передача данных на сервер](#13-передача-данных-на-сервер)

1. Как устроена система



Что делает MAPS:

- Принимает Excel-файлы с потребностями работ и остатками склада
- Автоматически распределяет материалы по 5 слоям (списания → склад → поставки → дефицит)
- Аварийные работы всегда получают материалы первыми
- Выдаёт Excel-отчёт с полным разбором

Что такое туннель:

Cloudflare создаёт временную публичную ссылку вида <https://xxxx.trycloudflare.com>.

Через неё можно открыть MAPS с любого компьютера в интернете, не открывая порты. URL меняется при каждом перезапуске туннеля.

2. Запуск на локальном компьютере

Используется для разработки и тестирования. Не нужен сервер.

2.1 Первый запуск (один раз)

```
# Перейти в папку проекта
cd ~/dev/projects/python/PythonProject
# Активировать виртуальное окружение
# Виртуальное окружение — это изолированная папка с Python и всеми пакетами проекта.
# Без него пакеты могут конфликтовать с другими проектами на компьютере.
source .venv/bin/activate
# Убедиться что .env настроен (должен быть файл с настройками БД)
cat .env
```

Минимальный .env для локального запуска:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=maps_db
DB_USER=maps_user
DB_PASSWORD=ваш_пароль
ANTHROPIC_API_KEY=sk-ant-api03-...
```

2.2 Запуск сервера

```
# Из папки проекта с активированным .venv:
.venv/bin/uvicorn app.api.main:app --reload --port 8000
```

Флаг `--reload` означает: при изменении кода сервер перезапускается автоматически.

Удобно для разработки, не нужен на сервере.

После запуска вы увидите:

```
INFO:      Started server process [12345]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000
```

Откройте в браузере: <http://localhost:8000>

2.3 Остановить сервер

Нажмите `Ctrl+C` в терминале где запущен uvicorn.

2.4 Если порт уже занят

Ошибка: `ERROR: [Errno 98] address already in use`

```
# Найти и убить процесс на порту 8000
sudo fuser -k 8000/tcp
# После этого снова запустить сервер
```

```
.venv/bin/uvicorn app.api.main:app --reload --port 8000
```

3. Работа с сервером Oracle Cloud

Сервер работает постоянно. MAPS запускается автоматически при включении сервера.

3.1 Подключение к серверу

Способ 1 — Oracle Cloud Shell (рекомендуется, самый простой):

1. Откройте cloud.oracle.com в браузере
2. Нажмите иконку >_ в правом верхнем углу — откроется встроенный терминал
3. В терминале наберите:

```
ssh ubuntu@132.145.232.46
```

Вы окажетесь внутри сервера. Никаких паролей и ключей не нужно — Oracle Cloud Shell уже авторизован.

Способ 2 — с локального компьютера (SSH-ключ уже настроен):

```
ssh -i /home/dan/keys/oracle.pem ubuntu@132.145.232.46
```

Ключ лежит в /home/dan/keys/oracle.pem. Oracle Cloud не принимает пароли — только ключи.

Если видите Permission denied (publickey) — используйте Способ 1 (Cloud Shell).

3.2 Управление сервисом MAPS

После подключения к серверу вы управляете MAPS через systemctl:

```
# Проверить статус (работает ли MAPS)
sudo systemctl status maps
# Запустить MAPS (если остановлен)
sudo systemctl start maps
# Остановить MAPS
sudo systemctl stop maps
# Перезапустить (например после обновления кода)
sudo systemctl restart maps
```

Что означает вывод systemctl status:

- maps.service - MAPS — Material Allocation & Planning System
 - Active: active (running) ← ☒ всё хорошо, MAPS работает
 - Active: activating (auto-restart) ← ☐ упал, пытается перезапуститься
 - Active: failed ← ☐ упал и не может запуститься

3.3 Просмотр логов

Логи помогают понять почему MAPS упал:

```
# Последние 30 строк лога
sudo journalctl -u maps -n 30 --no-pager
# Логи в реальном времени (выход: Ctrl+C)
sudo journalctl -u maps -f
```

```
# Логи с момента последнего запуска
sudo journalctl -u maps -b --no-pager
```

3.4 Запуск Cloudflare туннеля

Туннель нужен чтобы открыть MAPS с вашего компьютера.

Важно: Сначала убедитесь что MAPS запущен (`sudo systemctl status maps`).

```
# Перейти в папку проекта (обязательно!)
cd ~/maps
# Запустить туннель в фоне
bash deploy/tunnel.sh --bg
# Получить публичную ссылку
cat /tmp/maps_tunnel.url
```

Вы получите ссылку вида `https://xxxx.trycloudflare.com`.

Откройте её в браузере — это и есть дашборд MAPS.

Если туннель не запускается или ссылка не открывается:

```
# Проверить что MAPS отвечает локально
curl http://localhost:8000/api/status
# Если MAPS не отвечает — смотрим логи
sudo journalctl -u maps -n 20 --no-pager
# Если порт занят — освобождаем
sudo fuser -k 8000/tcp
sudo systemctl restart maps
# Перезапустить туннель
bash deploy/tunnel.sh --bg
cat /tmp/maps_tunnel.url
```

4. Ежедневная работа: полный цикл

Типичный рабочий день выглядит так:

1. Открыть Cloud Shell → запустить туннель → открыть дашборд
2. Загрузить обновлённые Excel-файлы (потребности, склад, поставки)
3. Нажать "Запустить распределение"
4. Скачать Excel-отчёт
5. При необходимости — воспользоваться AI-анализом дефицита

Шаг 1 — Открыть дашборд

```
# На сервере (Cloud Shell — НЕ на локальном компьютере!):
cd ~/maps
bash deploy/tunnel.sh --bg
cat /tmp/maps_tunnel.url
# → Скопировать ссылку и открыть в браузере
```

Частые ошибки:

- Команды туннеля нужно запускать на сервере (через Cloud Shell или SSH), а не в локальном терминале. Если видите No such file or directory: /home/dan/maps — ВЫ В локальном терминале.

- URL туннеля виден сразу в выводе `tunnel.sh --bg`, даже если `cat` команда не сработала.

Шаг 2 — Загрузить данные

В дашборде справа — панель «Импорт файлов». Раскройте нужный раздел и выберите файл.

Рекомендуемый порядок загрузки:

1. Перечень работ ← загружать первым: устанавливает даты и наименования работ
2. Потребности
3. Аварийные работы (если есть)
4. Складские остатки
5. Поставки
6. Списания
7. Выдано не списано

После каждой загрузки появится статистика: сколько строк загружено и сколько ошибок.

Шаг 3 — Запустить распределение

Нажмите кнопку «Запустить распределение» в левой панели.

Кнопка станет неактивной, появится индикатор загрузки.

Через несколько секунд в карточке «Текущие результаты» появится статистика: потребности, строки распределения, дефицит.

Шаг 4 — Скачать отчёт

В карточке «Текущие результаты» нажмите «Скачать Excel-отчёт».

Браузер скачает файл вида `MAPS_отчёт_20260524_...xlsx`.

5. Загрузка данных подробно

Порядок важен!

Списания и «Выдано не списано» ссылаются на работы.

Если загрузить их до потребностей — строки будут пропущены с ошибкой «Работа не найдена».

Правильный порядок:

- | | |
|----------------------|--|
| 1. Перечень работ | ← устанавливает наименования и даты начала/окончания работ |
| 2. Потребности | ← создаёт материалы в БД; даты берёт из перечня работ |
| 3. Аварийные работы | ← добавляет аварийные работы (тот же формат) |
| 4. Складские остатки | ← загружает партии на склад |
| 5. Поставки | ← загружает ожидаемые поставки («в пути») |
| 6. Списания | ← ссылаются на работы из шага 2 |
| 7. Выдано не списано | ← ссылаются на работы из шага 2 |

Важно: Если загрузить потребности без перечня работ — работы создадутся без дат.

При распределении они уйдут в конец очереди (NULLS LAST).

Загрузите перечень работ и запустите распределение повторно — даты встанут на место.

Повторная загрузка (обновление данных)

Загружать файлы можно сколько угодно раз. Все типы файлов работают по принципу полной замены:

Файл	Поведение
Перечень работ	Upsert: обновляет наименование и даты для существующих работ, создаёт новые
Потребности	Старые плановые потребности удаляются, загружаются новые
Аварийные работы	Старые аварийные потребности удаляются, загружаются новые
Склад	Старые партии удаляются, загружаются новые
Поставки	Старые поставки удаляются, загружаются новые
Списания	Старые записи удаляются, загружаются новые
Выдано не списано	Старые записи удаляются, загружаются новые

Система всегда работает с актуальными данными — при загрузке потребностей все предыдущие данные (работы, потребности, результаты распределения) полностью удаляются и заменяются новыми. История не хранится.

Загрузка через curl (без браузера)

Можно загружать файлы из командной строки. Удобно для автоматизации:

```
# С локального компьютера (замените URL на ваш туннельный)
TUNNEL=https://xxxx.trycloudflare.com
curl -X POST $TUNNEL/api/import/works -F "file=@путь/к/перечень_работ.xlsx"
curl -X POST $TUNNEL/api/import/requirements -F "file=@путь/к/файлу.xlsx"
curl -X POST $TUNNEL/api/import/emergency -F "file=@путь/к/аварийные.xlsx"
curl -X POST $TUNNEL/api/import/stock -F "file=@путь/к/склад.xlsx"
curl -X POST $TUNNEL/api/import/supplies -F "file=@путь/к/поставки.xlsx"
curl -X POST $TUNNEL/api/import/writeoffs -F "file=@путь/к/списания.xlsx"
curl -X POST $TUNNEL/api/import/issued -F "file=@путь/к/выдано.xlsx"
```

Ответ при успехе:

```
{"status": "ok", "stats": {"works": 50, "requirements": 1200, "errors": 0}}
```

Если в поле errors не ноль — посмотрите логи сервера:

```
sudo journalctl -u maps -n 50 --no-pager | grep -i error
```

Форматы Excel-файлов

Названия колонок в файле должны совпадать с ожидаемыми. Если колонки названы иначе — строки будут пропущены без ошибки (просто не загрузятся).

Перечень работ (загружать первым):

Колонка	Обязательна	Пример
Код работы	☐	WO-001
Наименование работы	нет	Ремонт насосной станции НС-6
Тип работы	нет	Ремонт
Филиал	нет	Алматы
Подразделение	нет	Цех 1
Центр затрат	нет	СС-KZ01-001
Завод	нет	KZ01
Дата начала	нет	2026-06-01
Дата окончания	нет	2026-07-31
Приоритет	нет	1
Статус	нет	active

Даты начала и окончания берутся только из этого файла. В файле потребностей дат нет.

Работа считается завершённой если Дата окончания < сегодня — по ней применяются только слои 1–2 (списание + выдано), склад и поставки не распределяются.

Потребности и аварийные работы:

Колонка	Обязательна	Пример
Код работы	☐	WO-001
Тип работы	нет	Плановый ремонт
Филиал	нет	Алматы
Завод	нет	KZ01
Приоритет	нет	1
Статус	нет	active
Системный номер	☐	MAT-001
Наименование материала	нет	Кабель ВВГ 3х2,5 мм
Ед.изм	нет	м
Потребность	☐	100
Прогнозная цена	нет	1200

Складские остатки:

Колонка	Обязательна	Пример
Код склада	☐	WH-KZ01-001
Тип склада	нет	Основной
Филиал склада	☐	Алматы
Завод склада	☐	KZ01
Системный номер	☐	MAT-001
Наименование материала	☐	Кабель ВВГ 3х2,5 мм
Ед.изм	☐	м
Номер партии	нет	BATCH-001
Количество	☐	500
Стоимость за ед	нет	1200
Дата поступления	☐	2026-01-15

6. Запуск распределения и скачивание отчёта

Через дашборд

1. Нажмите «Запустить распределение»
2. Дождитесь статуса ☐ «завершено» (обычно 5–30 секунд)
3. Нажмите «Скачать Excel»

Через API (curl)

```
# Запустить распределение
curl -X POST http://localhost:8000/api/allocate
# Проверить текущий статус
curl http://localhost:8000/api/status
# Скачать отчёт текущей сессии (ID берём из maps show или maps status)
curl -O -J http://localhost:8000/api/export/20260524_083028_f8db0
```

Что означают числа в итогах сессии

```
{
  "total_requirements": 16,  ← сколько строк потребностей обработано
  "total_allocated": 19,    ← сколько строк результатов создано (может быть > requirements,
                             если материал берётся из нескольких партий)
  "total_deficit": 14       ← сколько позиций в дефиците
}
```

Структура Excel-отчёта

№	Лист	Что там
1	Распределение	Полная таблица: работа, материал, все слои в одной строке (склад/поставка/списание/дефицит)
2	Движение склада	Детально по партиям: что взято, откуда, сколько
3	Остатки складов	Что осталось на складе после распределения
4	Возможное перемещение	Материалы которых нет на своём складе, но есть на складе другого филиала
5	Обеспеченность	Сводная таблица по работам: стоимость потребности, покрытия и дефицита, % обеспеченности
6	В пути	Поставки («в пути»), распределённые на конкретные работы в этой сессии
7	Не распред. в пути	Строки поставок с остатком после распределения — материалы в пути, которые никуда не ушли

7. AI-функции

Для работы AI нужен `ANTHROPIC_API_KEY` в файле `.env`.

Без него всё остальное работает нормально, только AI-кнопки вернут ошибку 503.

Объяснение дефицита

В дашборде внизу — синий блок «AI-анализ дефицита»:

1. После запуска распределения список материалов с дефицитом заполняется автоматически
2. Выберите материал из выпадающего списка
3. Нажмите «Объяснить»
4. Claude объяснит на русском почему не хватило материала и что делать

AI анализирует текущую сессию — почему не хватило конкретного материала, что покрыло какой слой, чего не хватало. Если дефицита нет — список будет пустым.

8. Обновление кода на сервере

Когда разработчик пушит изменения в GitHub — их нужно применить на сервере вручную:

```
# На сервере (Cloud Shell):
cd ~/maps
# Скачать последние изменения из GitHub
git pull origin main
# Перезапустить MAPS чтобы изменения вступили в силу
sudo systemctl restart maps
# Убедиться что запустился нормально
sudo systemctl status maps
```

Если после `git pull` сервис не запустился — смотрите логи:

```
sudo journalctl -u maps -n 30 --no-pager
```

9. Типичные ошибки и как их исправить

9. Тестовые данные и переход на реальные

Где хранятся тестовые файлы

Тестовые Excel-файлы лежат в папке `data/test/` на локальном компьютере:

```
data/test/
├─ test_works.xlsx           — 13 работ W0-SCN (перечень с наименованиями и датами)
├─ test_requirements.xlsx    — 13 тестовых сценариев потребностей (без дат — берутся из works)
├─ test_emergency.xlsx       — 3 аварийных работы (AV-SCN-06, 07, 08)
├─ test_stock.xlsx           — тестовые остатки склада
├─ test_supplies.xlsx        — 6 строк поставок по 3 материалам (MAT-002, MAT-003, MAT-004)
├─ test_writeoffs.xlsx       — тестовые списания
└─ test_issued.xlsx          — тестовые «выдано не списано»
```

Что проверяют тестовые поставки

`test_supplies.xlsx` содержит 6 строк поставок, специально подобранных для проверки листов «В пути» и «Не распред. в пути»:

Договор	Поставщик	Филиал	Материал	Кол-во	Что проверяет
ДОГ-SCN08-AST	ТОО Тест-Астана	Астана	MAT-003	30	Частичное покрытие двух работ из одного договора
ДОГ-SCN08-ALM	ТОО Тест-Алматы	Алматы	MAT-003	100	Несовпадение филиала → остаток в «Не распред.»

Договор	Поставщик	Филиал	Материал	Кол-во	Что проверяет
ДОГ-SCN04-001	ТОО Тест-Снаб	Шымкент	МАТ-004	25	Частичное покрытие WO-SCN-04 (нужно 100)
ДОГ-ТЕСТ-А	ТОО Поставщик-А	Шымкент	МАТ-002	40	Три договора на один материал → FIFO
ДОГ-ТЕСТ-Б	ТОО Поставщик-Б	Шымкент	МАТ-002	40	
ДОГ-ТЕСТ-В	ТОО Поставщик-В	Шымкент	МАТ-002	40	Частичное — 20 ушло, 20 остаток

Ожидаемый результат после `maps allocate + maps export`:

Лист «В пути» — 6 строк (распределённые поставки):

- ДОГ-SCN08-AST → WO-SCN-03 (20 шт) + WO-SCN-08 (10 шт) — один договор, две работы
- ДОГ-SCN04-001 → WO-SCN-04 (25 шт, частично)
- ДОГ-ТЕСТ-А/Б/В → WO-SCN-11 (40+40+20 м, полностью)

Лист «Не распред. в пути» — 2 строки (остатки):

- ДОГ-SCN08-ALM МАТ-003 Алматы — 100 шт (работа WO-SCN-03 в Астане, филиалы не совпали)
- ДОГ-ТЕСТ-В МАТ-002 — 20 шт остаток из 40

Важно: Папка `data/test/` добавлена в `.gitignore` — файлы не попадают в `git`.

Это сделано чтобы случайно не запустить реальные данные клиентов.

Тестовые файлы хранятся только на локальном компьютере разработчика.

Проблема: тестовые и реальные данные смешались

Что происходит:

Вы загрузили реальные потребности (199 строк) поверх тестовых данных.

В БД остались и те и другие. При запуске распределения все 215 строк

конкурируют за один и тот же (тестовый) склад — реальные результаты некорректны.

Как определить что данные смешались:

- Количество потребностей неожиданно большое
- В отчёте видны коды работ вида WO-SCN-01, AV-SCN-06 (тестовые)
- Обеспечённость неожиданно низкая (почти всё в дефиците)

Решение — полная очистка и загрузка только реальных данных:

```
# На сервере: остановить MAPS
sudo systemctl stop maps
# Сбросить все данные через psql
sudo -u postgres psql -d maps_db -c "
```

```
TRUNCATE TABLE deficit_records, allocation_results, stock_movements,  
    allocation_sessions, issued_not_written_offs, write_offs,  
    supply_lines, supplies, stock_batches, warehouses,  
    requirements, works, materials  
RESTART IDENTITY CASCADE;  
"
```

```
# Запустить снова  
sudo systemctl start maps
```

После этого загружайте только реальные файлы через дашборд в правильном порядке:

1. Потребности (реальные)
2. Аварийные работы (реальные, если есть)
3. Складские остатки (реальные)
4. Поставки (реальные)
5. Списания (реальные)
6. Выдано не списано (реальные)

Как проверить что в БД нет тестовых данных

Тестовые работы имеют коды вида W0-SCN-01 ... W0-SCN-12, AV-SCN-06 ... AV-SCN-08.

```
# Быстрая проверка через maps status — смотрим на количество работ  
maps status  
# Подробная проверка через psql — видим реальные коды  
sudo -u postgres psql -d maps_db -c "  
SELECT kod_raboty, is_emergency FROM works ORDER BY kod_raboty LIMIT 30;  
"
```

Если видите строки с W0-SCN-* или AV-SCN-* — тестовые данные присутствуют.

Если тестовые работы нашлись — выполните очистку выше перед загрузкой реальных данных.

Частичная очистка: удалить только тестовые записи

Если реальные данные уже загружены и не хочется всё удалять — можно точно удалить только тестовые работы по паттерну кода.

Пример: удалить тестовые аварийные работы (AV-SCN-*)

Важно: несколько таблиц ссылаются на works через work_id. Удалять нужно строго по порядку — от дочерних таблиц к родительской. Если пропустить хоть одну — PostgreSQL выдаст ошибку FK и не даст удалить работу.

Полный правильный порядок:

```
# 1. stock_movements ссылается на works  
sudo -u postgres psql -d maps_db -c "DELETE FROM stock_movements WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'AV-SCN-');"  
# 2. allocation_results ссылается на works  
sudo -u postgres psql -d maps_db -c "DELETE FROM allocation_results WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'AV-SCN-');"  
# 3. deficit_records ссылается на works  
sudo -u postgres psql -d maps_db -c "DELETE FROM deficit_records WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'AV-SCN-');"  
# 4. requirements ссылается на works  
sudo -u postgres psql -d maps_db -c "DELETE FROM requirements WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'AV-SCN-');"
```

```
# 5. Теперь можно удалить сами работы
sudo -u postgres psql -d maps_db -c "DELETE FROM works WHERE kod_raboty LIKE 'AV-SCN%'"
# 6. Проверяем — должно быть 0
sudo -u postgres psql -d maps_db -c "SELECT COUNT(*) FROM works WHERE kod_raboty LIKE 'AV-SCN%'"
```

Пример: удалить все тестовые обычные работы (WO-SCN-*)

Тот же порядок, только паттерн другой:

```
sudo -u postgres psql -d maps_db -c "DELETE FROM stock_movements WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'WO-SCN%')"
sudo -u postgres psql -d maps_db -c "DELETE FROM allocation_results WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'WO-SCN%')"
sudo -u postgres psql -d maps_db -c "DELETE FROM deficit_records WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'WO-SCN%')"
sudo -u postgres psql -d maps_db -c "DELETE FROM requirements WHERE work_id IN (SELECT id FROM works WHERE kod_raboty LIKE 'WO-SCN%')"
sudo -u postgres psql -d maps_db -c "DELETE FROM works WHERE kod_raboty LIKE 'WO-SCN%'"
```

Как узнать какая таблица блокирует удаление:

Если получили ошибку вида:

```
ERROR: update or delete on table "works" violates foreign key constraint
"stock_movements_work_id_fkey" on table "stock_movements"
```

— в тексте ошибки написано точное название таблицы (stock_movements). Сначала удалите записи из неё, потом повторите удаление работы.

После удаления тестовых данных запустите распределение заново — результат будет корректным:

```
maps allocate
maps show # проверить результат
```

Почему нельзя просто удалить через maps reset-data:

maps reset-data удаляет ВСЁ включая реальные данные и историю сессий. Точечное удаление через psql сохраняет реальные данные нетронутыми.

Как понять что очистка помогла:

Запустите maps show после нового maps allocate — если количество дефицита стало меньше, значит тестовые данные действительно мешали.

10. Работа с GitHub

Зачем нужен GitHub

GitHub — это облачное хранилище кода. Он выполняет три роли:

1. Резервная копия — код не потеряется если что-то случится с компьютером
2. История изменений — можно посмотреть что и когда изменили, откатиться назад
3. Мост между компьютерами — изменения сделанные на локальном компьютере попадают на сервер через GitHub

Локальный компьютер —git push→ GitHub —git pull→ Сервер Oracle Cloud

Репозиторий проекта

Адрес: <https://github.com/DaniarSher81212/maps>

Там хранится весь код MAPS. Ветка `main` — это актуальная рабочая версия.

Основные команды Git

Все команды выполняются в папке проекта на локальном компьютере:

```
cd ~/dev/projects/python/PythonProject
```

Посмотреть текущее состояние

```
git status
```

Покажет:

- Какие файлы изменены (но ещё не зафиксированы)
- Какие файлы добавлены в «очередь на коммит»
- На какой ветке вы находитесь

Пример вывода:

```
On branch main
Changes not staged for commit:
  modified:   app/services/import_service.py  ← файл изменён
Untracked files:
  docs/ИНСТРУКЦИЯ.md                        ← новый файл, ещё не добавлен
```

Посмотреть что именно изменилось в файле

```
git diff app/services/import_service.py
```

Покажет построчно: что было (красным со знаком -), что стало (зелёным со знаком +).

Добавить файлы в «очередь на коммит»

```
# Добавить конкретный файл:
git add app/services/import_service.py
# Добавить несколько файлов:
git add app/services/import_service.py docs/ИНСТРУКЦИЯ.md
# Добавить всё что изменилось (осторожно — может добавить лишнее):
git add .
```

Зафиксировать изменения (создать коммит)

```
git commit -m "описание что сделали"
```

Описание пишется на любом языке. Старайтесь описывать что изменили и зачем:

```
# Хорошо:
git commit -m "fix: повторный импорт склада больше не падает с FK-ошибкой"
git commit -m "docs: добавил раздел про GitHub в инструкцию"
# Плохо:
git commit -m "исправления"
git commit -m "update"
```

Отправить изменения на GitHub

```
git push origin main
```

После этого изменения видны на github.com и можно обновить сервер через `git pull`.

Получить изменения с GitHub (на сервере)

```
git pull origin main
```

Скачивает всё что было запущено с локального компьютера.

Полный цикл: внесли изменение → задеплоили

Это то что мы делаем каждый раз когда исправляем баг или добавляем новую функцию:

```
# 1. На локальном компьютере: смотрим что изменилось
git status
# 2. Добавляем изменённые файлы
git add имя_файла.py
# 3. Создаём коммит с описанием
git commit -m "описание изменения"
# 4. Пушим на GitHub
git push origin main
# 5. На сервере (Cloud Shell): скачиваем и перезапускаем
git pull origin main
sudo systemctl restart maps
sudo systemctl status maps
```

Просмотр истории коммитов

```
# Последние 10 коммитов (коротко)
git log --oneline -10
# Подробно с датами и авторами
git log -5
# Что изменилось в конкретном коммите (замените хэш на реальный)
git show 5fb80d3
```

Пример вывода `git log --oneline -5`:

```
d74ee8e docs: предупреждение — туннель только на сервере
fa52599 docs: обновлена таблица повторной загрузки
032c317 fix: полная замена при импорте потребностей и аварийных работ
8eb4081 docs: раздел 9 — тестовые vs реальные данные
5ed8779 docs: инструкция администратора для начинающих
```

Откат к предыдущей версии

Если после изменения что-то сломалось и нужно вернуть рабочую версию:

```
# Посмотреть историю и найти нужный коммит
git log --oneline -10
# Вернуть конкретный файл к состоянию из нужного коммита
git checkout 5fb80d3 -- app/services/import_service.py
# Зафиксировать откат
git commit -m "revert: откат import_service к версии до поломки"
git push origin main
```

```
# На сервере обновить
git pull origin main
sudo systemctl restart maps
```

Что НЕ попадает в GitHub

Файлы из `.gitignore` никогда не загружаются на GitHub:

Что	Почему
<code>.env</code>	Содержит пароли и API-ключи — никогда не публиковать
<code>data/test/</code>	Тестовые Excel-файлы — только на локальном компьютере
<code>data/exports/</code>	Сгенерированные отчёты — временные файлы
<code>__pycache__/</code>	Скомпилированный Python — генерируется автоматически
<code>.venv/</code>	Виртуальное окружение — у каждого своё

Важно: Если случайно добавили `.env` в `git` — немедленно смените все пароли и API-ключи. Даже удалённый из `git` файл остаётся в истории.

Как посмотреть код на GitHub

Зайдите на <https://github.com/DaniarSher81212/maps> и вы увидите:

- Code — все файлы проекта
 - Commits — история всех изменений
 - Actions — (будущее) CI/CD автоматические тесты
-

11. Типичные ошибки и как их исправить

Ошибка 1: «Permission denied (publickey)» при SSH

Что видите:

```
ubuntu@132.145.232.46: Permission denied (publickey)
```

Причина: Oracle Cloud не принимает пароли. Нужен SSH-ключ.

Решение: Используйте Oracle Cloud Shell вместо обычного SSH:

- Откройте cloud.oracle.com
 - Нажмите иконку `&gt_` в правом верхнем углу
 - В браузерном терминале наберите: `ssh ubuntu@132.145.232.46`
-

Ошибка 2: Дашборд отдаёт «Not Found»

Что видите: Белая страница с текстом «Not Found. The requested URL was not found on the server.»

Причина: MAPS запущен, но приложение упало при старте и маршруты не зарегистрированы.

Диагностика и решение:

```
# На сервере: смотрим логи
sudo journalctl -u maps -n 30 --no-pager
# Возможная ошибка 1: ModuleNotFoundError: No module named 'anthropic'
# Решение: установить пакет вручную
cd ~/maps && .venv/bin/pip install anthropic
sudo systemctl restart maps
# Возможная ошибка 2: address already in use (порт занят)
# Решение:
sudo fuser -k 8000/tcp
sleep 2
sudo systemctl restart maps
# Возможная ошибка 3: --workers 2 (многопроцессный режим не работает на Oracle Cloud)
# Проверить:
grep ExecStart /etc/systemd/system/maps.service
# Если там есть --workers 2 — убрать:
sudo sed -i '/--workers/d' /etc/systemd/system/maps.service
sudo systemctl daemon-reload
sudo systemctl restart maps
```

Ошибка 3: «Попытка соединения не удалась» при открытии туннельного URL

Что видите: Браузер говорит «не удаётся подключиться к серверу».

Причина: Туннель остановился (например при закрытии Cloud Shell) или MAPS упал.

Решение:

```
# На сервере:
# Шаг 1: убедиться что MAPS работает
sudo systemctl status maps
curl http://localhost:8000/api/status
# Шаг 2: если MAPS не работает — починить (см. Ошибку 2 выше)
# Шаг 3: перезапустить туннель
cd ~/maps
bash deploy/tunnel.sh --bg
cat /tmp/maps_tunnel.url
# → Скопировать новый URL и открыть в браузере
```

Ошибка 4: FK violation при повторном импорте

Что видите:

```
{"detail": "ForeignKeyViolation: update or delete on table..."}
```

Причина (историческая): В старых версиях MAPS при повторном импорте склада/поставок возникала ошибка FK. Исправлено в версии после 24.05.2026 —

повторный импорт работает корректно.

Если всё же получаете эту ошибку (старая версия):

```
# На сервере: обновить код
cd ~/maps && git pull origin main && sudo systemctl restart maps
```

Ошибка 5: Сервис циклически перезапускается (counter at 800+)

Что видите:

```
Active: activating (auto-restart)
maps.service: Scheduled restart job, restart counter is at 884
```

Причина: MAPS постоянно падает при старте. Systemd пытается его перезапустить.

Решение:

```
# Шаг 1: смотрим причину в логах
sudo journalctl -u maps -n 20 --no-pager
# Шаг 2: останавливаем сервис чтобы он перестал циклиться
sudo systemctl stop maps
# Шаг 3: освобождаем порт если занят
sudo fuser -k 8000/tcp
# Шаг 4: тестируем запуск вручную — видим реальную ошибку
cd ~/maps
.venv/bin/uvicorn app.api.main:app --host 0.0.0.0 --port 8000
# → Смотрим что выводится в терминал
# Шаг 5: исправляем проблему (по тексту ошибки — см. другие разделы)
# Шаг 6: запускаем через systemd снова
sudo systemctl start maps
sudo systemctl status maps
```

Ошибка 6: «address already in use» — порт занят

Что видите в браузере: Туннель открыт и URL есть, но страница отдаёт «Not Found».

Что видите в логах:

```
ERROR: [Errno 98] error while attempting to bind on address ('0.0.0.0', 8000): address already in use
maps.service: Scheduled restart job, restart counter is at 9732.
```

Причина: Предыдущий процесс uvicorn не завершился (например при systemctl stop воркеры зависли).

MAPS циклически перезапускается, каждый раз упираясь в занятый порт. Счётчик restart counter может быть очень большим — это нормально, главное освободить порт.

Cloudflare туннель при этом «успешно» подключается к старому зависшему процессу на порту 8000 — поэтому URL есть, но возвращается 404 вместо дашборда.

Решение:

```
# Найти и убить всё что занимает порт 8000
sudo fuser -k 8000/tcp
# Подождать 2 секунды и перезапустить
sleep 2
```

```
sudo systemctl restart maps
# Убедиться что запустился (должно быть active (running))
sudo systemctl status maps
# Проверить что API отвечает
curl http://localhost:8000/api/status
```

После этого обновите страницу в браузере — дашборд откроется.

Ошибка 7: Импорт прошёл, но часть строк пропущена

Что видите:

```
{"status": "ok", "stats": {"writeoffs": 3, "errors": 0, "skipped": 10}}
```

В поле skipped не ноль.

Причина: Строки списаний или «выдано не списано» ссылаются на работы, которых нет в БД.

Решение: Сначала загрузите потребности (/api/import/requirements), потом повторите загрузку файла который давал skipped.

Ошибка 8: ModuleNotFoundError: No module named 'main' при вызове maps

Что видите:

```
Traceback (most recent call last):
  File "/home/ubuntu/maps/.venv/bin/maps", line 3, in <module>
    from main import app
ModuleNotFoundError: No module named 'main'
```

Причина: Скрипт .venv/bin/maps устарел и содержит старую точку входа. Это происходит когда pip install -e . не пересоздаёт скрипт — старый файл остаётся нетронутым.

Решение — полная переустановка:

```
cd ~/maps
git pull origin main          # сначала получить свежий код
pip uninstall maps -y        # удалить старый пакет (и скрипт)
pip install -e .              # установить заново — скрипт пересоздастся
maps show                    # проверить что работает
```

Почему pip install -e . недостаточно:

Pip не перезаписывает уже существующий скрипт .venv/bin/maps если считает что пакет уже установлен.

pip uninstall удаляет скрипт, после чего pip install создаёт его заново с правильной точкой входа.

Важно: всегда делайте git pull ДО переустановки — иначе установится старая версия кода.

Общий чеклист «что-то не так» (раздел 10)

Если непонятно что происходит — пройдите по этому списку сверху вниз:

```
# 1. MAPS запущен?
sudo systemctl status maps
# 2. Порт свободен?
sudo ss -tlnp | grep 8000
# 3. Приложение отвечает?
curl http://localhost:8000/api/status
# 4. Что в логах?
sudo journalctl -u maps -n 30 --no-pager
# 5. Пакеты установлены?
cd ~/maps && .venv/bin/pip list | grep -E "anthropic|fastapi|uvicorn"
# 6. .env есть и содержит нужные переменные?
cat ~/maps/.env
# 7. PostgreSQL работает?
sudo systemctl status postgresql
```

12. Работа через командную строку (CLI)

Что такое CLI и зачем он нужен

CLI (Command Line Interface) — это управление системой через команды в терминале вместо браузера и мышки.

Когда CLI удобнее дашборда:

Задача	Дашборд	CLI
Загрузить файл мышкой	☑ удобно	можно
Загрузить 5 файлов подряд скриптом	неудобно	☑ удобно
Посмотреть статус не открывая браузер	—	☑ одна команда
Запустить распределение по расписанию (cron)	☑ нельзя	☑ легко
Посмотреть детали сессии в терминале	—	☑
Загрузить файл на сервере без браузера	☑ нельзя	☑

Как активировать CLI

Перед использованием команды `maps` нужно активировать виртуальное окружение:

```
# На локальном компьютере
cd ~/dev/projects/python/PythonProject
source .venv/bin/activate
# На сервере
cd ~/maps
source .venv/bin/activate
```

После активации в начале строки появится `(.venv):`

```
(.venv) dan@computer:~/dev/projects/python/PythonProject$
```

Теперь можно писать команды. Чтобы выйти из виртуального окружения:

```
deactivate
```

Если команда `maps` не найдена или выдаёт ошибку `ModuleNotFoundError`

После `git pull` (обновления кода с GitHub) нужно переустановить пакет, чтобы точка входа `maps` обновилась:

```
cd ~/maps
source .venv/bin/activate
pip install -e .
```

Что делает `pip install -e .`:

- `-e (editable)` — устанавливает пакет в «режиме разработки»: изменения в коде сразу видны без переустановки
- `.` — текущая папка (читает `pyproject.toml` для настроек)
- Регистрирует точку входа `maps` — создаёт скрипт `.venv/bin/maps` который запускает `main.py`

Когда нужно запускать `pip install -e .`:

- При первоначальной установке (делает `setup.sh` автоматически)
- После `git pull` если добавлены новые зависимости в `pyproject.toml`
- Если команда `maps` внезапно перестала работать с ошибкой `ModuleNotFoundError`

Если `pip install -e .` не помогло — скрипт `.venv/bin/maps` устарел, нужна полная переустановка:

```
pip uninstall maps -y
pip install -e .
```

Пока не переустановили — можно использовать `python main.py` ВМЕСТО `maps`:

```
python main.py status
python main.py show
python main.py allocate
```

Правильный порядок обновления CLI на сервере

После каждого `git pull` который затрагивает `main.py` или `pyproject.toml`:

```
cd ~/maps
git pull origin main
pip uninstall maps -y      # удаляем старый скрипт
pip install -e .           # создаём новый скрипт с актуальной точкой входа
sudo systemctl restart maps # перезапускаем веб-сервер
```

Если изменялись только файлы в `app/` или `docs/` — `pip uninstall/install` не нужен, достаточно `sudo systemctl restart maps`.

Справка по всем командам

Список всех команд с описанием

```
maps --help
# Справка по конкретной команде
maps import-stock --help
maps allocate --help
maps export --help
```

Полный справочник команд

maps init — инициализация базы данных

Выполняется один раз при первом запуске системы. Создаёт все таблицы в PostgreSQL. Безопасно вызывать повторно — существующие данные не удаляются.

```
maps init
```

Вывод при успехе:

```
✓ Подключение успешно
✓ Все таблицы созданы/проверены
Система готова к работе! Следующий шаг:
  maps import-requirements data/sample_requirements.xlsx
```

maps status — состояние базы данных

Показывает сколько записей в каждой таблице.

Удобно для быстрой проверки: загружены ли данные, сколько потребностей, есть ли результаты распределения.

```
maps status
```

Пример вывода:

MAPS — Статус

Количество записей в таблицах

Таблица / Группа	Записей
Работы (всего)	50
Аварийные работы	3
Материалы	120
Потребности	1 200
...	...

Если в таблице 0 записей — значение показывается серым: нужно загрузить данные.

maps show — детали текущей сессии

Показывает подробную информацию о текущей (единственной) сессии:

- Общую статистику
- Итоги по каждому слою распределения (сколько строк и общее количество материала)

- Топ-10 материалов в дефиците

maps show

Пример вывода:

Текущая сессия распределения
ID: 20260524_143055_a1b2c
Статус: completed
Запущена: 2026-05-24 14:30:55
Завершена: 2026-05-24 14:31:02

Обработано потребностей 16

Строк распределения 19

Позиций дефицита (К за купу) 14

Итоги по слоям

Слой	Строк	Количество всего
Слой 1 – Фактические списания	2	100.00
Слой 2 – Выдано не списано	1	50.00
Слой 3 – Склад	2	200.00
Слой 5 – К за купу (дефицит)	14	3 200.00

Топ-10 материалов в дефиците

Системный номер	Наименование	Дефицит	Ед. изм
MAT-003	Труба стальная 50 мм	1 500.00	м
MAT-007	Кабель ВВГ 3х2,5 мм	300.00	м

Для экспорта в Excel: maps export

Если распределение ещё не запускалось — команда выведет подсказку.

Как читать результаты maps show

Обработано потребностей 199 ← сколько строк из таблицы requirements

Строк распределения 190 ← сколько строк результата создано

Позиций дефицита (К за купу) 46 ← сколько позиций не хватило материала

Итоги по слоям показывают откуда взялись материалы:

Источник	Слой	Что значит
spisanie	Слой 1	Уже фактически списано в SAP
vydano	Слой 2	Выдано физически, но не списано
sklad	Слой 3	Взято со склада своего завода/филиала
postavka	Слой 4	Зарезервировано из ожидаемых поставок

Источник	Слой	Что значит
vozmozhnoe_sklad	Доп.	Анализ: можно взять с другого склада (перемещение)

Строки `vozmozhnoe_sklad` не уменьшают дефицит — это только подсказка для анализа.

Реальный дефицит (К закупу) — это то что не покрыто ни одним слоем.

Топ дефицитных материалов — список материалов которых не хватает больше всего.

Это приоритет для закупки: начинайте с верхней строки.

`maps import-*` — загрузка Excel-файлов

Все команды импорта работают одинаково: принимают путь к файлу, опционально — номер или имя листа.

```
# Загрузить потребности (обязательный первый шаг)
maps import-requirements data/requirements.xlsx
# Загрузить аварийные работы
maps import-emergency data/emergency.xlsx
# Загрузить складские остатки
maps import-stock data/stock.xlsx
# Загрузить поставки
maps import-supplies data/supplies.xlsx
# Загрузить фактические списания (Слой 1)
maps import-writeoffs data/writeoffs.xlsx
# Загрузить «Выдано не списано» (Слой 2)
maps import-issued data/issued.xlsx
```

Если данные на листе с именем, а не на первом листе:

```
maps import-requirements data/requirements.xlsx --sheet "Потребности"
maps import-requirements data/requirements.xlsx -s "Потребности"
```

Вывод при успехе:

Результат импорта: Потребности

Показатель		
works	50	← создано работ
requirements	1200	← загружено потребностей
materials	120	← создано/обновлено материалов
skipped	0	← пропущено (жёлтым если > 0)
errors	0	← ошибок (красным если > 0)

`maps allocate` — запуск распределения

Запускает 5-слойный алгоритм по всем загруженным данным. Создаёт новую сессию с автоматически сгенерированным ID вида `YYYYMMDD_nnmss_xxxxx`.

```
# Запустить с автоматическим ID
maps allocate
# Запустить с пользовательским ID (удобно для сценариев)
maps allocate --session "июнь_2026_план_A"
```



```
maps allocate -s "тест_без_аварийных"
```

Вывод:

✓ Распределение завершено!
ID сессии: 20260524_143055_a1b2c
Результаты распределения

Обработано потребностей	16
Строк распределения	19
Позиций дефицита	14

← красным если > 0

Для экспорта в Excel:
maps export

maps import-works — загрузка перечня работ

Загружает наименования и даты начала/окончания работ. Выполнять перед import-requirements.

```
# Загрузить перечень работ
maps import-works data/works.xlsx
# Если данные на именованном листе
maps import-works data/works.xlsx --sheet "Перечень работ"
```

Вывод при успехе:

Результат импорта: Перечень работ

created	50
updated	3
errors	0

← новых работ создано

← существующих обновлено

maps export — экспорт отчёта в Excel

Создаёт Excel-файл с 7 листами по результатам текущей сессии. SESSION_ID не нужен — всегда экспортируется единственная текущая сессия.

```
# Сохранить в папку по умолчанию (data/exports/)
maps export
# Сохранить в конкретную папку
maps export --output /tmp/отчёты
maps export -o ~/Downloads
```

Вывод:

✓ Файл создан: data/exports/MAPS_отчёт_20260524_143055_a1b2c.xlsx
Структура отчёта:

1. «Распределение» — главная широкая таблица
2. «Движение склада» — детально по партиям
3. «Остатки складов» — что осталось
4. «Возможное перемещение» — анализ других филиалов
5. «Обеспеченность» — % покрытия по стоимости в разбивке по работам
6. «В пути» — поставки, распределённые на работы в этой сессии
7. «Не распред. в пути» — поставки с остатком после распределения

maps serve — запуск веб-сервера

Запускает FastAPI с дашбордом. Использовать когда нужен браузерный интерфейс.

```
# Запустить на порту 8000 (по умолчанию)
maps serve
# Запустить на другом порту
maps serve --port 9000
# Режим разработки (перезапуск при изменении кода)
maps serve --reload
```

maps reset-data — полный сброс данных

ВНИМАНИЕ: необратимая операция! Удаляет все данные из всех таблиц.

Используется для перехода с тестовых данных на реальные или для повторного тестирования.

```
maps reset-data
```

Полный рабочий цикл через CLI (без браузера)

Вот как можно выполнить полный рабочий цикл только в терминале:

```
# Активировать окружение
source .venv/bin/activate
# Проверить состояние (загружены ли данные?)
maps status
# Загрузить данные (порядок важен!)
maps import-works          data/works.xlsx          # первым — даты и наименования
maps import-requirements  data/requirements.xlsx
maps import-emergency     data/emergency.xlsx       # если есть
maps import-stock         data/stock.xlsx
maps import-supplies      data/supplies.xlsx        # если есть
maps import-writeoffs     data/writeoffs.xlsx       # если есть
maps import-issued        data/issued.xlsx          # если есть
# Запустить распределение
maps allocate
# Посмотреть детали текущей сессии
maps show
# Скачать отчёт
maps export
# Открыть файл (Linux)
xdg-open data/exports/MAPS_отчёт_*.xlsx
```

Загрузка данных на сервере через CLI (без браузера)

Если нужно загрузить файлы прямо на сервере (например через SFTP или скрипт):

```
# На сервере (Cloud Shell):
cd ~/maps
source .venv/bin/activate
# Загружаем файлы которые уже скопированы на сервер
maps import-requirements /home/ubuntu/data/requirements.xlsx
maps import-stock        /home/ubuntu/data/stock.xlsx
```

```
maps allocate
maps show      # проверить результат
maps export    # создать Excel-файл
```

После экспорта файл лежит в `~/maps/data/exports/`. ID текущей сессии видно в `maps show` — его можно использовать для скачивания через браузерный URL:

```
https://xxxx.trycloudflare.com/api/export/20260524_XXXXX_XXXXX
```

Автоматизация: запуск по расписанию (cron)

Если нужно запускать распределение автоматически каждый день в 06:00:

```
# Открыть редактор crontab
crontab -e
# Добавить строку (Ctrl+O для сохранения, Ctrl+X для выхода):
0 6 * * * cd /home/ubuntu/maps && .venv/bin/maps allocate >> /tmp/maps_cron.log 2>&1
```

Что означает `0 6 * * *`: каждый день в 06:00.

Результат запуска (успех или ошибка) пишется в `/tmp/maps_cron.log`.

Проверить лог:

```
tail -20 /tmp/maps_cron.log
```

13. Передача данных на сервер

Есть три способа загрузить Excel-файлы на сервер. Выбирайте в зависимости от ситуации.

Способ	Когда использовать	Сложность
Дашборд (браузер)	Разовая загрузка, удобно для нетехнических пользователей	☐ просто
curl (API)	Загрузка нескольких файлов скриптом, автоматизация	☐☐ средне
SCP (прямая копия)	Нет браузера, нужна работа с файлами на сервере через CLI	☐☐☐ сложнее

Способ 1 — Через дашборд (браузер)

Самый простой способ. Не требует никаких дополнительных инструментов.

Шаги:

1. На сервере запустите туннель:

```
cd ~/maps
bash deploy/tunnel.sh --bg
cat /tmp/maps_tunnel.url
```

2. Откройте полученный URL в браузере (вида `https://xxxx.trycloudflare.com`)

3. В правой панели дашборда раскройте нужный раздел (например «Потребности»)
4. Нажмите «Выбрать файл», выберите Excel-файл на вашем компьютере, нажмите «Загрузить»
5. Дождитесь сообщения об успешной загрузке со статистикой

Ограничения:

- Туннель должен быть запущен и URL — актуальным
- Каждый файл загружается отдельно через интерфейс
- Размер файла ограничен настройками сервера (обычно до 100 МБ)

Способ 2 — Через curl (API запросы)

Загрузка файлов напрямую через HTTP API с вашего компьютера. Удобно когда нужно загрузить несколько файлов подряд или автоматизировать процесс.

Требования: туннель должен быть запущен, curl установлен (есть по умолчанию на Linux и Mac).

Разовая загрузка одного файла

```
# Сначала сохраните URL туннеля в переменную
TUNNEL=https://xxxx.trycloudflare.com # замените на реальный URL
# Загрузить потребности
curl -X POST $TUNNEL/api/import/requirements \
  -F "file=@/home/dan/Downloads/requirements.xlsx"
```

Флаг -F "file=@путь" означает: отправить файл как form-data (так же как через браузер).

Знак @ перед путём — указание что это файл, а не текст.

Ответ при успехе:

```
{"status": "ok", "stats": {"works": 63, "requirements": 199, "errors": 0}}
```

Загрузка всех файлов скриптом

Создайте файл load_data.sh на локальном компьютере:

```
#!/bin/bash
# load_data.sh — загружает все файлы на сервер через API
TUNNEL=https://xxxx.trycloudflare.com # ← вставьте актуальный URL туннеля
DATA_DIR=/home/dan/Downloads/maps_data # ← папка с вашими Excel-файлами
echo "Загрузка перечня работ..."
curl -s -X POST $TUNNEL/api/import/works \
  -F "file=@$DATA_DIR/works.xlsx" | python3 -m json.tool
echo "Загрузка потребностей..."
curl -s -X POST $TUNNEL/api/import/requirements \
  -F "file=@$DATA_DIR/requirements.xlsx" | python3 -m json.tool
echo "Загрузка аварийных работ..."
curl -s -X POST $TUNNEL/api/import/emergency \
  -F "file=@$DATA_DIR/emergency.xlsx" | python3 -m json.tool
echo "Загрузка складских остатков..."
curl -s -X POST $TUNNEL/api/import/stock \
  -F "file=@$DATA_DIR/stock.xlsx" | python3 -m json.tool
echo "Загрузка поставок..."
curl -s -X POST $TUNNEL/api/import/supplies \
```

```
-F "file=@$DATA_DIR/supplies.xlsx" | python3 -m json.tool
echo "Загрузка списаний..."
curl -s -X POST $TUNNEL/api/import/writeoffs \
-F "file=@$DATA_DIR/writeoffs.xlsx" | python3 -m json.tool
echo "Загрузка выдано не списано..."
curl -s -X POST $TUNNEL/api/import/issued \
-F "file=@$DATA_DIR/issued.xlsx" | python3 -m json.tool
echo "Готово! Запустите распределение через дашборд или:"
echo "  curl -X POST $TUNNEL/api/allocate"
```

Запуск скрипта:

```
chmod +x load_data.sh # дать права на выполнение (один раз)
./load_data.sh        # запустить
```

Флаг `-s` (silent) убирает прогресс-бар curl, | python3 -m json.tool красиво форматирует JSON-ответ.

Запуск распределения через curl

```
# Запустить распределение
curl -X POST $TUNNEL/api/allocate
# Проверить текущий статус
curl -s $TUNNEL/api/status | python3 -m json.tool
# Скачать отчёт (ID берём из maps show на сервере)
curl -O -J $TUNNEL/api/export/20260524_143055_alb2c
# -O — сохранить в файл, -J — использовать имя файла из заголовка ответа
```

Ограничения:

- Нужен активный туннель
- URL туннеля меняется при каждом перезапуске — нужно обновлять в скрипте
- Если туннель закрылся в процессе — загрузка прервётся

Способ 3 — Через SCP (прямая копия файлов)

SCP (Secure Copy) — копирование файлов напрямую на сервер по SSH, без туннеля и браузера.

После копирования файлы появляются на сервере и можно загружать их через CLI (maps import-*).

Требования: настроенный SSH-ключ для подключения к серверу.

SSH-ключ для подключения к серверу

SSH-ключ уже настроен и лежит по пути:

```
/home/dan/keys/oracle.pem
```

Подключение к серверу с локального компьютера:

```
ssh -i /home/dan/keys/oracle.pem ubuntu@132.145.232.46
```

Флаг `-i` (identity) — указывает путь к файлу ключа.

`.pem` — формат ключа Oracle Cloud (приватный ключ, никому не передавать).

Копирование файлов через SCP

SCP использует тот же ключ что и SSH — добавьте `-i /home/dan/keys/oracle.pem`:

```
# Скопировать один файл на сервер
scp -i /home/dan/keys/oracle.pem \
    /home/dan/Downloads/requirements.xlsx \
    ubuntu@132.145.232.46:~/maps/data/
# Скопировать несколько файлов сразу
scp -i /home/dan/keys/oracle.pem \
    requirements.xlsx stock.xlsx supplies.xlsx \
    ubuntu@132.145.232.46:~/maps/data/
# Скопировать всю папку с файлами
scp -i /home/dan/keys/oracle.pem -r \
    /home/dan/Downloads/maps_data/ \
    ubuntu@132.145.232.46:~/maps/data/
```

Флаг `-r` (recursive) — копировать папку со всем содержимым.

После копирования на сервере (через Cloud Shell или SSH):

```
cd ~/maps
source .venv/bin/activate
# Загрузить скопированные файлы через CLI
maps import-requirements data/requirements.xlsx
maps import-stock data/stock.xlsx
maps import-supplies data/supplies.xlsx
maps allocate
maps show
```

Скачать файл с сервера на локальный компьютер

```
# Скачать Excel-отчёт с сервера
scp -i /home/dan/keys/oracle.pem \
    ubuntu@132.145.232.46:~/maps/data/exports/MAPS_отчёт_*.xlsx \
    ~/Downloads/
```

Удобный алиас (чтобы не писать ключ каждый раз)

Добавьте в файл `~/.ssh/config` на локальном компьютере:

```
Host maps-server
    HostName 132.145.232.46
    User ubuntu
    IdentityFile /home/dan/keys/oracle.pem
```

После этого можно использовать короткие команды:

```
ssh maps-server # подключиться
scp maps-server:~/maps/data/exports/*.xlsx ~/Downloads/ # скачать отчёт
scp requirements.xlsx maps-server:~/maps/data/ # загрузить файл
```

Ограничения:

- Ключ `/home/dan/keys/oracle.pem` должен быть только на локальном компьютере
- Права на файл ключа должны быть 600: `chmod 600 /home/dan/keys/oracle.pem`
- Нельзя использовать через Oracle Cloud Shell — только с локального компьютера

Сравнение способов на практике

Ситуация

Рекомендуемый способ

Загрузить 1-2 файла вручную	Дашборд (Способ 1)
Загрузить 6 файлов каждый день	curl-скрипт (Способ 2)
Загрузить файлы без интернета/VPN	SCP (Способ 3)
Автоматически в 6:00 каждое утро	curl-скрипт + cron (Способ 2)
Переслать файл коллеге без SSH-ключа	Дашборд (Способ 1)

Полный рабочий цикл через SCP + CLI (пошагово с реальными примерами)

Это полный разбор того как загрузить файлы через SCP и импортировать их через CLI. Все команды проверены на практике.

Шаг 1 — Подготовить папку на сервере

Перед первой загрузкой создайте папку для данных на сервере.

Выполняется на сервере (через Cloud Shell):

```
mkdir -p ~/maps/data/test      # папка для тестовых данных
mkdir -p ~/maps/data/real      # папка для реальных данных
```

Флаг `-p` — создать папку вместе со всеми родительскими папками если их нет.

Шаг 2 — Скопировать файлы через SCP

Выполняется на локальном компьютере (не в Cloud Shell!).

Важно: команда должна быть в одну строку — вставка многострочной команды с `\` может не сработать.

```
# Скопировать папку с тестовыми данными целиком
scp -i /home/dan/keys/oracle.pem -r /home/dan/dev/projects/python/PythonProject/data/test/ ubuntu@132.145.232.4
```

Вывод при успехе — каждый файл с прогресс-баром:

```
test_stock.xlsx      100% 5793    61.7KB/s   00:00
test_supplies.xlsx   100% 5656    59.6KB/s   00:00
test_writeoffs.xlsx  100% 5315    58.7KB/s   00:00
test_emergency.xlsx  100% 5610    59.0KB/s   00:00
test_requirements.xlsx 100% 6555    67.6KB/s   00:00
test_issued.xlsx     100% 5178    54.1KB/s   00:00
```

Частая ошибка — двойная вложенность папок:

Если папка назначения уже существовала до SCP, файлы могут попасть на уровень глубже:

```
~/maps/data/test/test/test_stock.xlsx ← лишний уровень test/
```

Проверьте:

```
ls ~/maps/data/test/
```

Если видите `test` (папку внутри папки) — переместите файлы:

```
mv ~/maps/data/test/test/* ~/maps/data/test/ && rmdir ~/maps/data/test/test
```

Шаг 3 — Импортировать файлы через CLI

Выполняется на сервере. Порядок важен — потребности первыми:

```
cd ~/maps
source .venv/bin/activate
maps import-works          data/test/test_works.xlsx
maps import-requirements  data/test/test_requirements.xlsx
maps import-emergency     data/test/test_emergency.xlsx
maps import-stock         data/test/test_stock.xlsx
maps import-supplies      data/test/test_supplies.xlsx
maps import-writeoffs     data/test/test_writeoffs.xlsx
maps import-issued       data/test/test_issued.xlsx
```

Каждая команда выводит таблицу с результатом. Пример для потребностей:

Импорт потребностей из: data/test/test_requirements.xlsx
Результат импорта: Потребности

Показатель	Значение	
works	0	← 0 потому что работы уже есть в БД
requirements	13	← загружено 13 потребностей
errors	0	← ошибок нет ☐

Что означает `works = 0`:

Работы создаются при первой загрузке. При повторной загрузке потребностей для тех же работ — новые работы не создаются, считается 0. Это нормально.

На что смотреть:

- `errors = 0` — всё загружено без ошибок ☐
- `skipped > 0` — часть строк пропущена (обычно потому что работа не найдена — нужно сначала загрузить потребности)
- `errors > 0` — смотреть логи: `sudo journalctl -u maps -n 30 --no-pager`

Шаг 4 — Запустить распределение и проверить результат

```
maps allocate
maps show
```

Пример вывода `maps show` после чистого тестового прогона:

```
Текущая сессия распределения
ID: 20260524_060515_81943
Статус: completed
Запущена: 2026-05-24 06:05:15
Завершена: 2026-05-24 06:05:15

Обработано потребностей      16
Строк распределения          19
Позиций дефицита (К закупу)  8
```

Как читать результат:

- 16 потребностей = 13 обычных + 3 аварийных ☐
- 19 строк распределения > 16 потребностей — нормально, один материал может покрываться из нескольких источников
- 8 дефицит — 8 позиций к закупу (не хватило материала)

Шаг 5 — Скачать отчёт

Экспортировать на сервере (ID берётся автоматически из текущей сессии)

```
maps export
```

Скачать на локальный компьютер через SCP (с локального компьютера)

```
scp -i /home/dan/keys/oracle.pem ubuntu@132.145.232.46:~/maps/data/exports/MAPS_отчёт_*.xlsx ~/Downloads/
```

Или скачать через браузер (если туннель запущен, ID из `maps show`):

https://xxxx.trycloudflare.com/api/export/20260524_060515_81943